Written Skills Alter Validation Behavior Without Consistently Improving Repository Repair

Wang Qichen

Abstract

Natural-language skills prescribe how a tool-using coding agent should inspect, edit, and validate a repository. Whether such procedures improve task success across models remains uncertain because behavioral changes and verified outcomes are often conflated. We introduce SkillTraj, a frozen task pool and execution harness with external verifiers and complete tool-trajectory records. We evaluated 18 held-out repository-repair tasks across six model configurations in 870 scheduled runs, comparing a no-skill prompt (C0) with a fixed baseline skill (C1); a rewritten skill (v1) was explored on one weak target model. C1–C0 verifier-pass differences were small and inconsistent across models. For `qwen3.5:9b`, C1 changed task-paired post-edit validation by −25.0 pp while verifier pass changed by +1.9 pp. The exploratory v1 condition changed validation by +45.0 pp relative to C1, while verifier pass changed by 0.0 pp. A four-cell development cross-tab likewise showed that several target behaviors occurred in both passing and failing runs. The task pool is synthetic and small; task-level minimum detectable differences were approximately 9.0–11.6 pp for the evaluated three-repeat models. These results describe this task pool and do not establish that written skills generally lack benefit.

1 Introduction

Natural-language procedures are a low-cost way to specify how a coding agent should inspect a repository, edit files, and validate its changes. A procedure can alter what an agent does without changing what a verifier accepts. Evaluations therefore need to measure both the final artifact and the trajectory that produced it.

This study asks whether a fixed written skill changes repository-repair success across models when the task specification, tools, verifier, and schedule are held constant. We constructed a small task pool with deterministic external checks, then compared a no-skill condition (C0) with a fixed baseline skill (C1). We also examined whether target behaviors separated passing from failing trajectories in development data and whether a revised skill could change those behaviors in a held-out subset.

The study makes three contributions:

1. A task pool and harness with frozen external verifiers, admission checks, and complete tool-trajectory records.
2. A locked cross-model C0/C1 evaluation over 18 held-out tasks, with task-level paired analysis and a verifier-based primary outcome.
3. Descriptive evidence that process behavior can move without a corresponding change in verified outcome, including a four-cell behavior–outcome cross-tab.

2 Related Work

Interactive language-model work has studied trajectories that alternate reasoning and actions. ReAct made such interleaving explicit and evaluated it across reasoning and interactive decision tasks [Yao et al., 2023]. Our study takes the trajectory as an object of measurement, while asking a narrower question about fixed procedural instructions in repository repair.

Software-engineering agents are commonly evaluated through task completion against executable tests. SWE-bench uses real GitHub issues and repository-level changes with test-based evaluation

[Jimenez et al., 2024]; SWE-agent studies how an agent–computer interface changes software-engineering task performance and agent behavior [Yang et al., 2024]. Our task pool is synthetic and much smaller, but freezes each specification and verifier so the C0/C1 prompt contrast can be examined alongside detailed action traces.

Instruction-following evaluations have developed verifiable constraints for model outputs and agent settings. IFEval focuses on automatically checkable instructions [Zhou et al., 2023]; AgentIF evaluates longer, constraint-rich instructions in agentic applications, including tool-related constraints [Qi et al., 2025]. These lines motivate separating instruction compliance from task success. A directly comparable, multi-model study of persistent repository-repair skills with held-out task-level inference remains to be established; this positioning is incomplete pending a targeted literature review [TOD].

3 Methods

3.1 Task pool and study design

SkillTraj contains small repository-repair tasks with a task statement, a buggy seed, fixtures, and an external verifier. Development data were divided into diagnosis and validation subsets; the held-out set comprised 12 `test_in` tasks from Skills A/B and six `test_out` tasks from Skill C. Admission checks required positive fixtures to pass and buggy or shortcut fixtures to fail the verifier.

The primary comparison paired C0 (no skill) with C1 (the frozen v0 skill). Six model configurations were scheduled: `qwen3.5:9b`, `gemma4:26b`, `qwen3.8:27b`, `qwen3-coder-next`, `llama3.1:8b`, and `gpt-5.6-sol`. The first five used runner r2; `gpt-5.6-sol` was executed through Codex and is reported as an uncontrolled reference anchor. `qwen3.5:9b` had five repetitions per task and all other C0/C1 configurations had three. Two qwen-only exploratory conditions were also run: a revised skill condition (v1) on the 12 `test_in` tasks and a fixed call-32 reminder on all 18 tasks. The full schedule contained 870 runs.

The schedule and outcome protocol were specified in a hash-locked, timestamped plan before the held-out evaluation. The study was not externally preregistered. Conditions were interleaved using fixed per-model schedules. No task, verifier, fixture, or skill asset was changed during evaluation.

3.2 Execution and outcomes

Runner r2 recorded provider responses, parsed tool calls, command results, file edits, sampling metadata, terminal flags, and verifier outputs. It requested temperature 0.5, a seed when supported, up to 40 tool calls, a 65,536-token context budget, 120-second command timeouts, and a 1,200-second process-group timeout; metadata records requested and observed settings. Retry-exhausted provider errors were excluded from the primary pass-rate numerator and denominator. Hard timeouts and step-limit runs remained failures in the denominator. We report provider-error sensitivity with those runs counted as failures.

The primary outcome was the frozen Boolean `verifier.json.passed`, independent of the agent's final message. Terminal indicators such as `final_empty`, `no_edit`, and `timeout` were analyzed as non-exclusive process measures. A run with a final empty message could still pass the verifier.

3.3 Statistical analysis

The task was the sampling unit; repetitions estimated within-task variability. C0/C1 contrasts were task-paired and summarized by a fixed-seed, 10,000-draw paired bootstrap. Individual proportions use Wilson intervals. Wilson intervals describe single proportions and were not used as tests of between-condition differences. For process summaries, we report medians and interquartile ranges across task-level summaries, together with run-level occurrence rates where relevant.

Using observed within-task Bernoulli variance, approximate minimum detectable differences at two-sided $\alpha = 0.05$ and 80% power ranged from 9.0 to 11.6 pp across the controlled three-repeat models. This is a design-level detectability bound, not an observed treatment effect.

Table 1: Verifier pass for the locked core conditions. The sampling unit for the contrast is the task; intervals are 95% paired-bootstrap intervals over 18 tasks.

	Verifier pass, runs		Task-paired C1–C0
Model	C0	C1	Difference (pp)
`qwen3.5:9b`	43/89 (48.3%)	45/89 (50.6%)	+1.9 [−6.9, 10.6]
`gemma4:26b`	33/54 (61.1%)	36/54 (66.7%)	+5.6† [0.0, 14.8]
`qwen3.8:27b`	45/54 (83.3%)	46/54 (85.2%)	+1.9 [−3.7, 7.4]
`qwen3-coder-next`	33/54 (61.1%)	34/54 (63.0%)	+1.9 [−7.4, 13.0]
`llama3.1:8b`	0/54 (0.0%, floor)	0/54 (0.0%, floor)	0.0 [0.0, 0.0] (floor)
`gpt-5.6-sol`[a]	44/54 (81.5%)	44/54 (81.5%)	0.0 [−5.6, 5.6]

[a] Codex-run reference anchor; sampling and tool surface were not aligned to runner r2.
† The lower paired-bootstrap bound is exactly 0.0. Provider errors are excluded from primary denominators; the qwen denominators are 89 of 90 scheduled runs per condition.

Table 2: Verifier pass conditional on final-message status for held-out C0/C1 runs. The unit is a scheduled run; denominators include protocol provider-error records for this descriptive cross-tab (n = 504).

Model	All runs	Final empty	Pass if empty	Pass if non-empty
`qwen3.5:9b`	180	105 (58.3%)	50/105 (47.6%)	39/75 (52.0%)
`gemma4:26b`	108	2 (1.9%)	1/2 (50.0%)	68/106 (64.2%)
`qwen3.8:27b`	108	0 (0.0%)	0/0 (NA)	91/108 (84.3%)
`qwen3-coder-next`	108	7 (6.5%)	3/7 (42.9%)	64/101 (63.4%)

These are descriptive behavior rates; all scheduled records, including protocol provider-error records, are retained in these conditional denominators.

4 Results

4.1 Cross-model verifier pass

All 870 scheduled run directories contained metadata and a Boolean verifier result. The controlled C0/C1 results were small and heterogeneous (table 1, fig. 1). `gemma4:26b` had a +5.6 pp task-paired difference, while `qwen3.5:9b`, `qwen3.8:27b`, and `qwen3-coder-next` each differed by +1.9 pp. `llama3.1:8b` remained at a complete floor. The GPT reference anchor had identical marginal pass counts under C0 and C1, but its sampling was not controlled through r2. By split, `qwen3.5:9b` changed by +5.0 pp on `test_in` and −4.2 pp on `test_out`; `qwen3.8:27b` and `qwen3-coder-next` changed direction across the two splits. One controlled task was all-zero and none was all-one across the model panel, limiting information from binary pass rates (table 6).

4.2 Final-message indicator and pass

In the `qwen3.5:9b` C0/C1 runs, 50 of 105 runs with `final_empty` passed (47.6%), compared with 39 of 75 runs without `final_empty` (52.0%). These conditional proportions describe output habits and do not separate passing from failing runs in this sample.

4.3 Mechanical failure counts

The weak-target and stronger-model failure cohorts contain 98 and 97 verifier-failing runs, respectively. The table reports non-exclusive terminal flags; a run can contribute to multiple columns.

Table 3: Non-exclusive mechanical failure flags among verifier-failing runs.

Cohort	*n*	No edit	Empty	Timeout	Compile	Step limit
Weak development diagnosis	8	0	3	0	0	0
Weak target, held-out C0	46	4	27	1	1	5
Weak target, held-out C1	44	2	27	1	1	8
Weak target, combined	98	6	57	2	2	13
Three stronger models, held-out	97	3	5	0	0	5

A run may contribute to more than one flag. The stronger group combines `gemma4:26b`, `qwen3.8:27b`, and `qwen3-coder-next`. Named failed verifier checks are summarized separately in the released mechanical taxonomy.

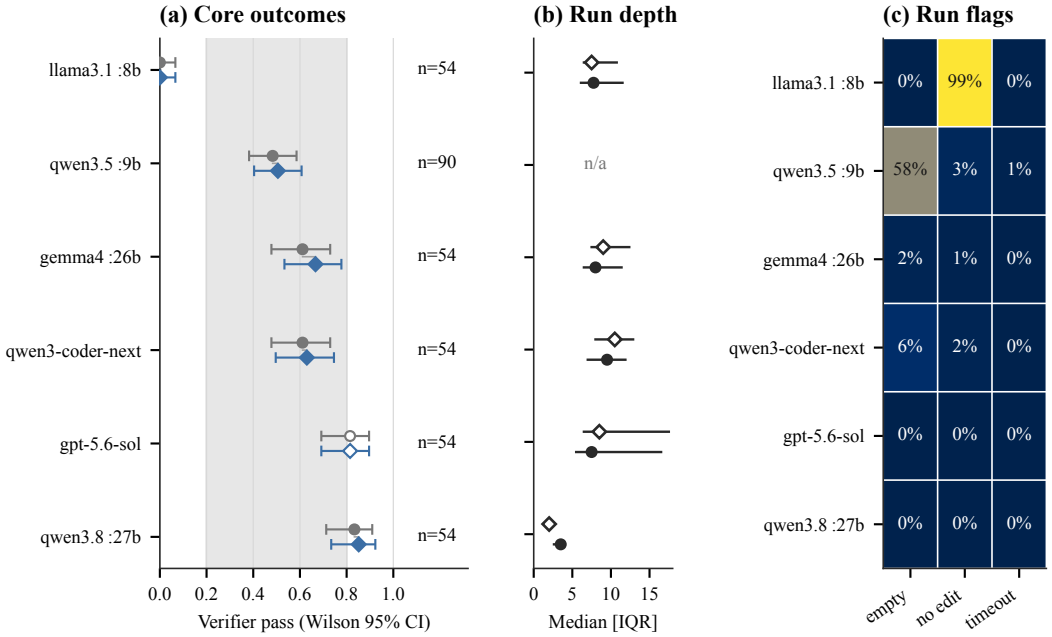


Figure 1: Held-out task-pool results for the six model configurations. In panel (a), gray circles denote C0 and blue diamonds C1; pass rates use runs and Wilson 95% intervals, while paired contrasts use tasks (18 task pairs per controlled model). Panel (b) shows medians and IQRs (filled circles: tool calls; open diamonds: assistant turns); panel (c) shows run-flag rates. Row-end (n) is scheduled runs (18 tasks by repetitions). GPT is an uncontrolled reference anchor.

4.4 Process behavior and verified outcome

For `qwen3.5:9b`, the task-paired strict validation-intent rate was 25.0 points lower under C1 than C0 (18 tasks; 95% interval $[-38.9, -9.4]$ pp). In the exploratory test-in comparison, v1 was 45.0 points higher than C1 (12 tasks; 95% interval $[26.7, 63.3]$ pp). The corresponding verifier-pass differences were +1.9 pp for C1–C0 (18 tasks; $[-6.9, 10.6]$) and 0.0 pp for v1–C1 (12 tasks; $[-11.7, 13.3]$). These contrasts describe behavioral and outcome differences; they do not establish mediation.

4.5 Strong-versus-weak process summaries

The strong comparison group contains `gemma4:26b`, `qwen3.8:27b`, and `qwen3-coder-next`; the weak target is `qwen3.5:9b`. Table 4 reports run-level medians and IQRs, with task-level summaries for binary process rates. Parentheses show run-level occurrence rates. Under r2, each parsed tool result is one tool call, so the step and call summaries coincide.

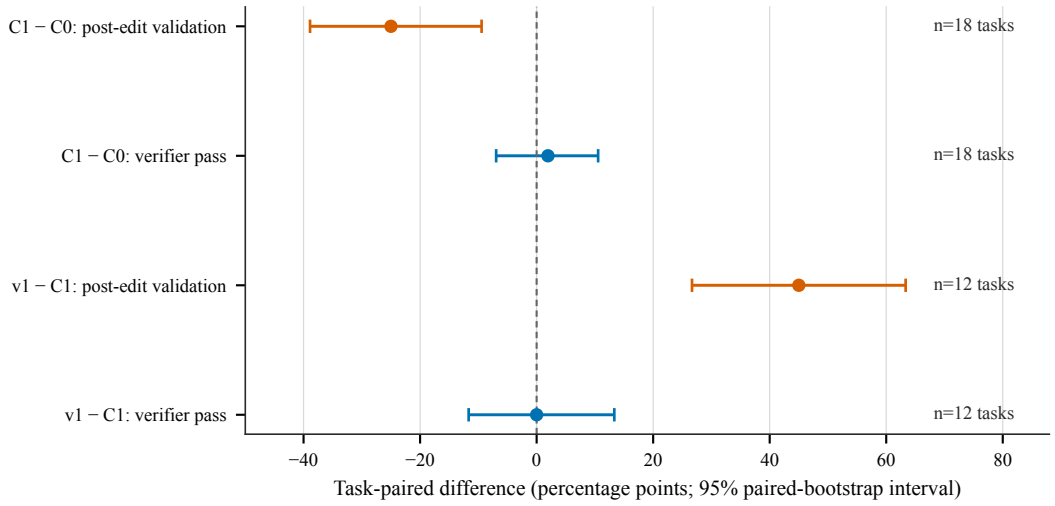


Figure 2: Task-paired qwen contrasts for validation behavior and verifier pass. Intervals are 95% paired-bootstrap intervals; the resampling unit is the task (n = 18 for C1–C0 and n = 12 for v1–C1). v1 is exploratory and limited to `test_in`.

Table 4: Strong and weak cohort process summaries under the core conditions. Values are median [IQR]; binary rates include run-level percentages in parentheses.

Cohort	Condition	n	Steps/calls	Turns	Intent	Success	Rewrites	Timeout
Strong	C0	162	8 [7,11]	9 [7.2,12]	100 [0,100]% (68.5%)	100 [100,100]% (93.8%)	0 [0,75] (25.3%)	0 [0,0]%
Strong	C1	162	9 [7,12]	9 [7,12]	100 [0,100]% (71.6%)	100 [100,100]% (97.5%)	0 [0,0] (17.9%)	0 [0,0]%
Weak	C0	89/90	8 [6,14]	9 [7,15]	100 [0,100]% (64.0%)	100 [100,100]% (86.5%)	0 [0,100] (40.4%)	0 [0,0]% (1.1%)
Weak	C1	89/90	7 [5,19]	8 [6,20]	0 [0,100]% (38.2%)	100 [0,100]% (65.2%)	0 [0,100] (31.5%)	0 [0,0]% (1.1%)

Values are median [IQR] across runs; parenthetical values are run-level rates. Intent is a command matching the frozen validation-intent rule; success is any successful command after the last edit. Under r2, one parsed tool result is one call, so steps and calls coincide. Strong includes three models; weak includes `qwen3.5:9b`. Weak usable runs exclude one retry-exhausted provider error per condition.

4.6 Development behavior–outcome cross-tab

The four-cell cross-tab uses only the development diagnosis subset, with mechanically defined behaviors and verifier outcomes. Reading before editing occurred in every cell. Validation and rereading were not less common in failing cells, and repeated-write avoidance was similar across cells (table 5, fig. 3). The weak pass and weak fail cells contain only 10 and 8 runs.

4.7 Mechanical failure classification

Among 98 verifier-failing weak-target runs from development diagnosis and held-out C0/C1, terminal flags were non-exclusive: 57 had an empty final message, 13 reached the step limit, two timed out, and two had a compile error. In the three stronger model configurations' held-out C0/C1 failures (n = 97), five reached the step limit and five had empty final messages. Failed verifier checks also included hidden-boundary and verifier-exception categories; details and counts are in the appendix. The sampled earlier judge packets cover none of these 98 weak-target failures, so the classifications in fig. 4 are mechanical rather than judge-derived.

Human–judge agreement was assessed on 24 packets; agreement statistics and their limits are reported in the appendix. Judge labels are used as conclusion evidence only for file corruption, tool misuse, and missed constraints; validation and looping claims rely on mechanical indicators.

4.8 Evaluation pipeline

The evaluation freezes tasks and verifiers, interleaves conditions, records tool trajectories, and verifies final workspaces before task-paired analysis (Figure 5).

Table 5: Mechanical behavior rates in the development diagnosis subset. Values are count/usable runs (percentage); cells are observational strata.

Behavior	Strong pass	Strong fail	Weak pass	Weak fail
Read before edit	47/47 (100.0%)	6/6 (100.0%)	10/10 (100.0%)	8/8 (100.0%)
Validate after edit	33/47 (70.2%)	6/6 (100.0%)	3/10 (30.0%)	5/8 (62.5%)
Avoid repeated full-file write	36/47 (76.6%)	5/6 (83.3%)	8/10 (80.0%)	6/8 (75.0%)
Reread after edit	12/47 (25.5%)	3/6 (50.0%)	2/10 (20.0%)	4/8 (50.0%)

Strong comprises three fixed models; weak is `qwen3.5:9b`. One retry-exhausted run is excluded. No interval is reported for these descriptive cells.

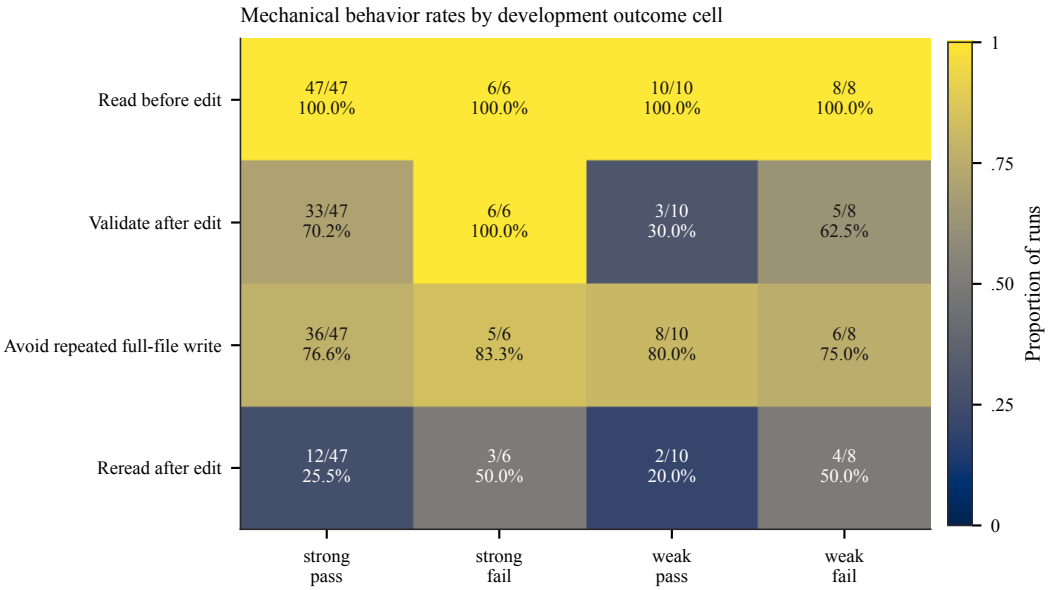


Figure 3: Development diagnosis behavior–outcome cross-tab. Cell sample sizes are 47, 6, 10, and 8 runs (strong-pass, strong-fail, weak-pass, weak-fail); labels show count and percentage. No inferential interval is shown.

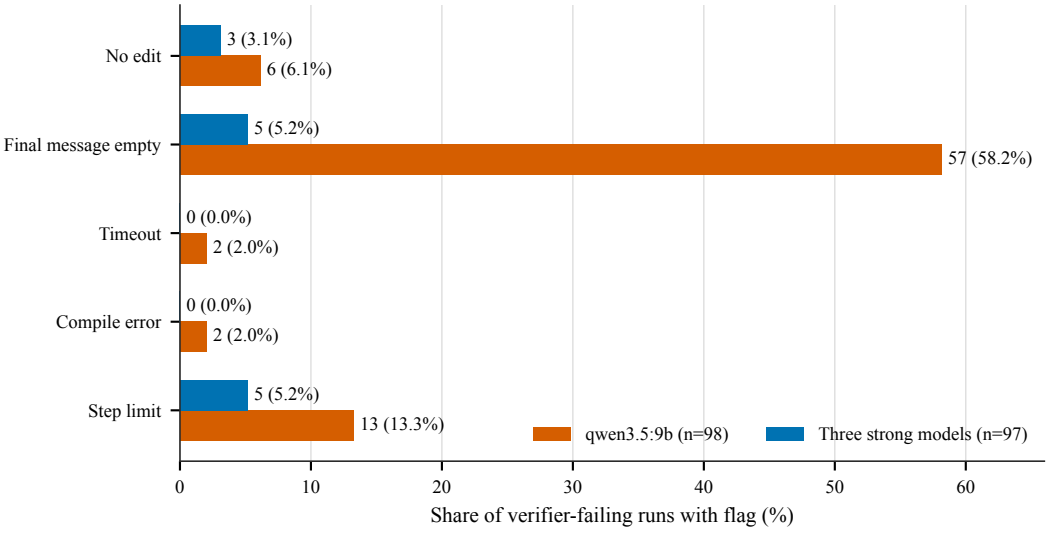


Figure 4: Mechanical terminal flags among verifier-failing runs. Bars show the percentage of runs in each cohort carrying a flag; flags are non-exclusive and counts are printed beside each bar. Denominators are 98 weak-target failures and 97 failures from three stronger models; no interval is shown.

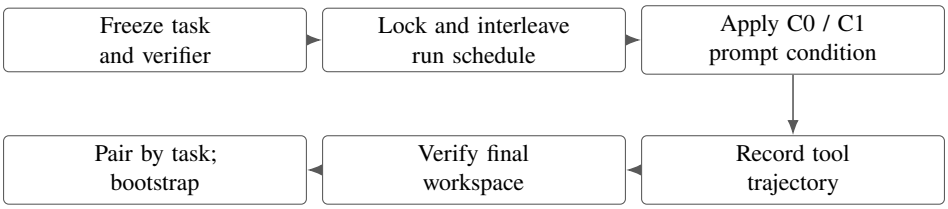


Figure 5: Evaluation pipeline schematic. The formal task pool contains 18 held-out tasks; the completed schedule contains 870 runs across six model configurations and core/exploratory conditions. The inferential unit for C0/C1 comparisons is the task.

5 Discussion

Across this task pool, changes in target behavior were more visible than changes in verifier pass. The revised skill shifted validation behavior substantially for the weak target in its exploratory subset, while the pass contrast remained centered at zero. The development cross-tab provides a complementary description: read-before-edit was universal in each cell, and validation, rereading, and rewrite avoidance did not uniquely mark passing runs. Together these observations caution against treating a checklist behavior as a proxy for task success.

The outcome-accounting audit also produced a methodological lesson. Earlier single-label summaries grouped some empty-final-message runs as failures even when the verifier passed. The primary analyses here therefore read success only from the verifier Boolean, while retaining terminal behaviors as separate, possibly overlapping indicators. Five retry-exhausted transport failures were excluded by the protocol; treating them as failures is the stated denominator sensitivity.

5.1 Limitations

The task pool is synthetic and self-authored, so it does not establish external validity for large real-world repositories. There are 18 held-out task units; repeated runs estimate within-task variability but do not increase the number of sampled tasks. The estimated minimum detectable differences of 9.0–11.6 pp indicate limited sensitivity to smaller changes. Floor and ceiling behavior further reduces information: `llama3.1:8b` was at a complete pass-rate floor, while one task was all-zero and no task was all-one across the controlled panel.

Repetition counts were unequal by design: `qwen3.5:9b` had five runs per task and other model configurations had three. `gpt-5.6-sol` was sampled through Codex, with a different tool surface and uncontrolled sampling, and is only a reference anchor. An initial batch of 104 `gpt-5.6-sol` usage-limit failures was overwritten after the owner requested a rerun; the overwritten archive was deleted, so potential selection differences between those attempts cannot be reconstructed. The current GPT cohort is not evidence about that discarded batch.

The development cross-tab is observational, and the v1 and call-32 reminder analyses are exploratory. Neither has a length-matched generic-prompt control. The reminder analysis also conditions on reaching its trigger point. Finally, human–judge comparison used one researcher annotator, incomplete blinding, and a small packet sample. The hash-locked, timestamped plan is not equivalent to independent preregistration. A larger task sample, matched prompt-length controls, and independently annotated trajectories would address these limits.

6 Broader Impact and Ethics

The task pool evaluates software-repair behavior in isolated workspaces and should not be interpreted as evidence of safe autonomous deployment. The paper reports model behavior and verifier outcomes, not judgments about individual users or developers. The human review component consisted of one researcher labeling anonymized trajectory packets; no external participant recruitment or personal data collection was performed.

7 Data and Code Availability

The paper's analysis scripts, de-identified metadata, verifier records, and figure sources are intended for release at `[repositoryURLtobeadded]`. Code is licensed under the MIT License and released data under CC BY 4.0. Raw provider responses containing reasoning are excluded from the public package.

A Split-Level Paired Results

Table 6: Task-paired C1–C0 verifier-pass differences by held-out split. Intervals are 95% paired-bootstrap intervals over tasks.

Model	`test_in`, 12 tasks	`test_out`, 6 tasks
`qwen3.5:9b`	+5.0 [−5.0, 13.3]	−4.2 [−23.3, 14.2]
`gemma4:26b`	+8.3† [0.0, 22.2]	0.0† [0.0, 0.0]
`qwen3.8:27b`	−2.8 [−8.3, 0.0]	+11.1† [0.0, 22.2]
`qwen3-coder-next`	−2.8 [−11.1, 5.6]	+11.1† [0.0, 33.3]
`llama3.1:8b`	0.0 [0.0, 0.0] (floor)	0.0 [0.0, 0.0] (floor)
`gpt-5.6-sol`[a]	0.0 [−8.3, 8.3]	0.0† [0.0, 0.0]

† The lower interval bound is exactly 0.0.
[a] Uncontrolled reference anchor, not pooled with r2 comparisons.

B Outcome-Accounting Audit

The legacy single-label outcome undercounted runs that passed the verifier but ended with an empty message. Reconstructing success from `verifier.json.passed` changed the earlier calibration queue from 102 to 110 passes among 175 usable runs, the original second queue from 8 to 18 among 28, the replacement-inclusive second queue from 12 to 24 among 36, and the development baseline/reminder queue from 25 to 48 among 72. These are outcome-accounting corrections, not performance changes.

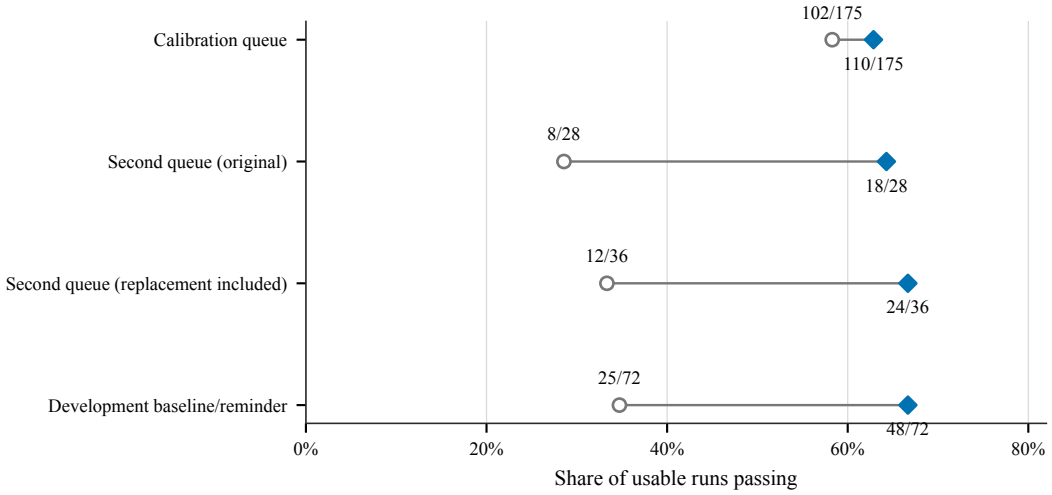


Figure 6: Legacy single-label and verifier-based pass counts across completed queues. Open circles denote legacy labels and diamonds denote verifier pass. The queue is the comparison unit; values are counts and no interval is shown. Usable-run denominators are 175, 28, 36, and 72, respectively.

C Human–Judge Agreement

Across 24 blinded packets, macro Cohen's κ over five estimable labels was 0.588 (bootstrap 95% interval 0.434–0.771); macro PABAK across all eight labels was 0.677 (0.552–0.781), and exact label-set agreement was 25.0%. Interpretation is limited by the single researcher annotator, incomplete blinding, and small sample.

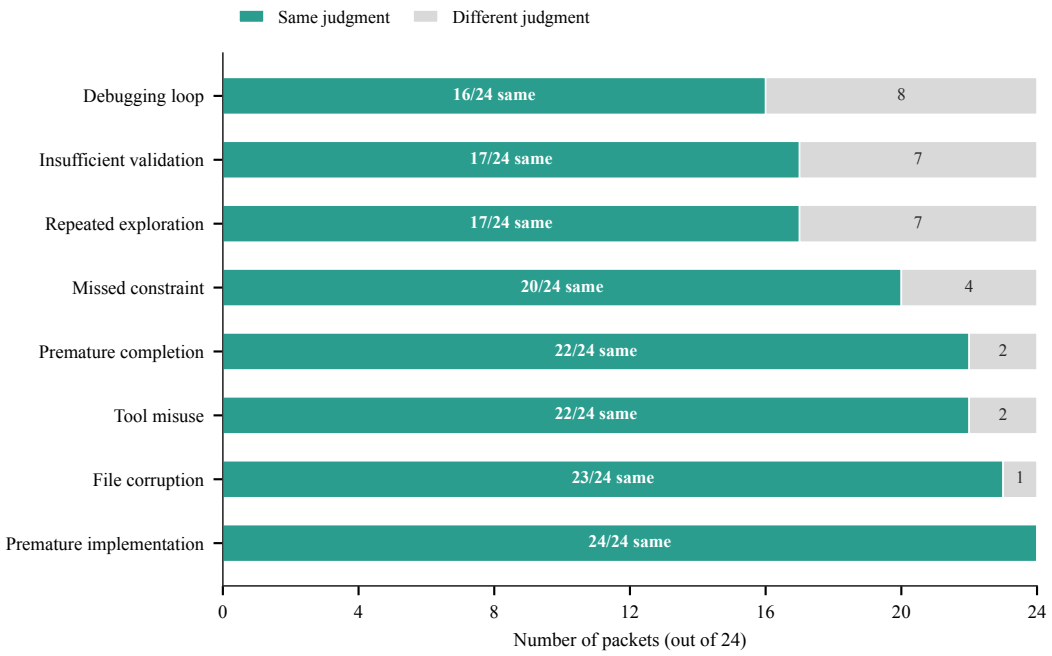


Figure 7: Raw human–judge agreement across 24 blinded packets. Each row shows the number of packets on which both raters assigned the same binary label (green) or different labels (gray). The annotation packet is the sampling unit. Raw agreement is shown for readability and can be high when a label is rare; chance-adjusted Cohen's κ and PABAK are reported in the accompanying appendix text.

References

Citation needed: controlled evaluations of persistent written agent skills in repository repair. Author to verify and replace before submission. Placeholder for a targeted literature review; not a bibliographic claim.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.

Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. AGENTIF: Benchmarking large language models instruction following ability in agentic scenarios. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/51bb3a8a33610a25aae074bfc51b1b1f-Abstract-Datasets_and_Benchmarks_Track.html.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601. URL https://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL https://arxiv.org/abs/2311.07911.